
CVM++

Stack-Based Virtual Machine and Custom Compiler

1. Introduction

Modern programming languages such as Python, Java, and JavaScript abstract away the internal execution mechanisms from the programmer. Internally, however, all such languages must transform human-readable source code into a sequence of machine-executable instructions through several well-defined stages.

This project, **CVM++**, builds this entire pipeline from scratch in C++. By constructing each component individually — lexer, parser, compiler, and virtual machine — the project makes visible the mechanisms that are ordinarily hidden inside production language runtimes.

The system supports a small but complete programming language that includes integer and boolean data types, arithmetic and comparison operators, variables, conditional statements, loops, and built-in input/output. Compiled programs are stored as binary bytecode files on disk, which the virtual machine can then load and execute independently of the original source code.

2. Problem Statement

Most developers write code in high-level languages without understanding how source text is translated into executable instructions. The aim of this project is to demonstrate the complete compilation pipeline by implementing each of its stages from first principles:

1. Lexical analysis — converting raw source text into a token stream.
2. Syntax analysis — arranging tokens into a structured Abstract Syntax Tree (AST).
3. Bytecode compilation — flattening the AST into a list of low-level instructions.

4. Virtual machine execution — running those instructions on a simulated stack machine.

The resulting system must support integer arithmetic, boolean values, variables, conditional branching, loops, and user input and output.

3. Objectives

The specific objectives of this project are as follows:

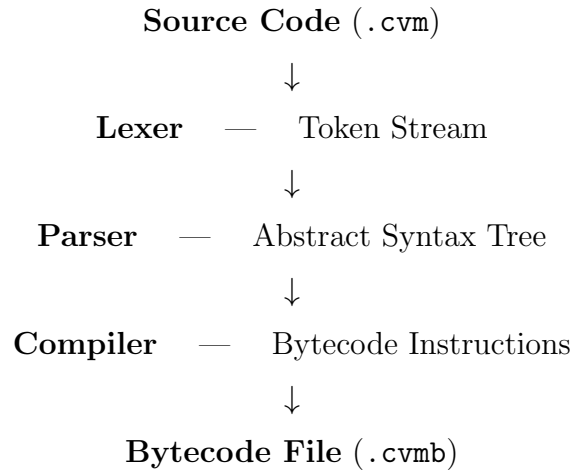
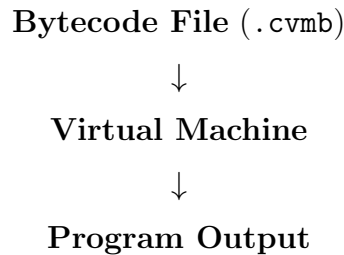
- Design a simple, well-defined programming language grammar.
- Implement a lexer that tokenizes source code strings.
- Implement a recursive descent parser that constructs an AST.
- Implement a compiler that emits bytecode instructions from the AST.
- Define a binary bytecode file format (`.cvmb`) and implement serialization and deserialization.
- Implement a stack-based virtual machine that executes bytecode.
- Provide a command-line interface supporting file compilation, bytecode execution, single-step execution, and an interactive REPL.

4. Technology Stack

- **Implementation Language:** C++ (C++17 standard)
- **Compiler:** g++ (GCC 7 or later)
- **Build Command:** Single-step g++ invocation; no build system required
- **Core Concepts Applied:**
 - Lexical analysis
 - Recursive descent parsing
 - Abstract Syntax Trees
 - Stack-based bytecode compilation
 - Virtual machine design
 - Binary file I/O

5. System Architecture

The overall pipeline of CVM++ consists of two independent phases, as illustrated below.

Phase 1 — Compilation**Phase 2 — Execution**

Separating these two phases means that a program needs to be compiled only once. The resulting `.cvmb` file can be distributed and executed on any machine running the CVM++ virtual machine, without access to the original source code.

5.1 Source File Structure

File	Responsibility
<code>lexer.h / lexer.cpp</code>	Tokenizes raw source text into a stream of typed tokens
<code>parser.h / parser.cpp</code>	Consumes the token stream and builds an AST
<code>compiler.h / compiler.cpp</code>	Walks the AST and emits bytecode instructions
<code>vm.h / vm.cpp</code>	Executes bytecode instructions on a stack-based machine
<code>bytecode_io.h / .cpp</code>	Serializes bytecode to <code>.cvmb</code> files and loads them back
<code>main.cpp</code>	Entry point; provides the CLI and interactive REPL

6. Lexer

The lexer is the first stage of the compilation pipeline. It reads a raw source string character by character and produces a flat list of typed tokens that the parser can consume.

6.1 Responsibilities

- Skip whitespace and newlines, which carry no syntactic meaning.
- Recognize numeric literals and convert them to `NUMBER` tokens.
- Distinguish keywords (`let`, `if`, `else`, `while`, `print`, `input`, `true`, `false`) from plain identifiers.
- Recognize single-character operators and the two-character operator `==`.
- Raise an error on any unrecognised character.

6.2 Supported Token Types

Category	Tokens
Literals	<code>NUMBER</code> , <code>BOOL_TRUE</code> , <code>BOOL_FALSE</code>
Identifiers	<code>IDENTIFIER</code>
Keywords	<code>LET</code> , <code>IF</code> , <code>ELSE</code> , <code>WHILE</code> , <code>PRINT</code> , <code>INPUT</code>
Arithmetic	<code>PLUS</code> , <code>MINUS</code> , <code>STAR</code> , <code>SLASH</code>
Comparison	<code>EQ (==)</code> , <code>LT (<)</code> , <code>GT (>)</code>
Symbols	<code>ASSIGN</code> , <code>SEMICOLON</code> , <code>LPAREN</code> , <code>RPAREN</code> , <code>LBRACE</code> , <code>RBRACE</code>
Control	<code>END_OF_FILE</code>

6.3 Example

Given the source line:

```
1 let x = 10 + 20;
```

The lexer produces the following token stream:

`LET` → `IDENTIFIER(x)` → `ASSIGN` → `NUMBER(10)` → `PLUS` → `NUMBER(20)` → `SEMICOLON`

7. Parser

The parser consumes the token stream produced by the lexer and arranges it into an Abstract Syntax Tree (AST), which represents the grammatical structure of the program.

7.1 Parser Strategy

The project uses a **recursive descent parser** — a top-down parsing technique in which each grammar rule corresponds directly to a C++ function. This approach is simple, readable, and well-suited to hand-written parsers.

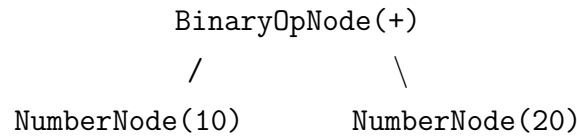
7.2 AST Node Types

Each construct in the language maps to a distinct node type:

Node Type	Represents
NumberNode	An integer literal
BoolNode	A boolean literal (<code>true</code> or <code>false</code>)
VarNode	A variable reference
BinaryOpNode	A binary expression (<code>left op right</code>)
AssignNode	A variable declaration or assignment
PrintNode	A <code>print</code> statement
InputNode	An <code>input</code> statement
IfNode	An <code>if/else</code> block
WhileNode	A <code>while</code> loop
ProgramNode	The root of the tree; holds all top-level statements

7.3 Example

The expression `10 + 20` is represented in the AST as:



8. Compiler

The compiler performs a single traversal of the AST and emits a flat, ordered list of bytecode instructions. Each AST node type maps to one or more opcodes from the instruction set.

8.1 Compilation Strategy

Expressions are compiled in a *post-order* fashion: both operands of a binary expression are compiled before the operator instruction is emitted, ensuring that by the time the operator executes, both values are already on the stack.

Control flow constructs (`if/else` and `while`) use *backpatching*: a jump instruction with a placeholder target is emitted first, and its operand is filled in once the true target address is known.

8.2 Instruction Set

Opcode	Operand	Description
PUSH	Integer value	Push a constant integer onto the stack
LOAD	Variable index	Push the value of a variable onto the stack
STORE	Variable index	Pop the stack top and store it in a variable
ADD	—	Pop two values; push their sum
SUB	—	Pop two values; push their difference
MUL	—	Pop two values; push their product
DIV	—	Pop two values; push their integer quotient
EQ	—	Pop two values; push 1 if equal, else 0
LT	—	Pop two values; push 1 if left < right, else 0
GT	—	Pop two values; push 1 if left > right, else 0
PRINT	—	Pop the stack top and print it
INPUT	Variable index	Read an integer from stdin into a variable
JUMP	Instruction index	Jump unconditionally to the given address
JUMP_IF_FALSE	Instruction index	Pop the stack top; jump if the value is 0
HALT	—	Stop execution

Table 1: CVM++ Bytecode Instruction Set

9. Bytecode File Format

Compiled programs are written to binary `.cvmb` files, which can be stored on disk and executed independently of the source code. The format is as follows:

Offset	Size	Field
0	4 bytes	Magic bytes: 'C', 'V', 'M', 'B'
4	1 byte	Format version (0x01)
5	4 bytes	Variable count (unsigned 32-bit, little-endian)
9	Variable	Variable table: [4-byte name length] [name bytes] per entry
n	4 bytes	Instruction count (unsigned 32-bit, little-endian)
$n + 4$	5 bytes each	Instructions: [1-byte opcode] [4-byte signed operand]

Table 2: CVM++ Bytecode File Format (`.cvmb`)

The magic bytes allow the loader to verify that a file is a valid `.cvmb` file before attempting to parse it, and the version byte provides a path for future format changes.

10. Virtual Machine

The virtual machine loads a `.cvmb` file from disk and executes its instructions. It operates on a stack-based architecture, meaning that all operands and results are passed through a central operand stack rather than through named registers.

10.1 Internal Components

- **Operand Stack** — a dynamic array of integers; the working memory for all computations.
- **Variable Storage** — a flat array, one slot per variable, indexed by the variable table.
- **Instruction Pointer (IP)** — an index into the instruction list; advances by one after each instruction, except for jump instructions.

10.2 Execution Cycle

The VM executes the following cycle repeatedly until a `HALT` instruction is reached:

1. Fetch the instruction at the current IP.
2. Decode its opcode.
3. Execute the corresponding operation (potentially reading from or writing to the stack or variable storage).
4. Advance the IP (or jump, for control-flow instructions).

11. Language Reference

11.1 Data Types

CVM++ supports two primitive types:

- **Integer** — a signed integer value (e.g. 0, 42, -7).
- **Boolean** — the literals `true` and `false`, stored internally as 1 and 0.

11.2 Operators

Operator	Meaning	Example
+	Addition	<code>a + b</code>
-	Subtraction	<code>a - b</code>
*	Multiplication	<code>a * b</code>
/	Integer division	<code>a / b</code>
==	Equality	<code>a == b</code>
<	Less than	<code>a < b</code>
>	Greater than	<code>a > b</code>

11.3 Variables

Variables are declared using `let` and may be reassigned without `let`:

```

1 let x = 10;
2 x = x + 1;

```

11.4 Conditional Statements

```

1 if (x < y) {
2     print(x);
3 } else {
4     print(y);
5 }

```

11.5 Loops

```

1 while (x < 10) {
2     print(x);
3     x = x + 1;
4 }

```

11.6 Input and Output

```

1 input(x);
2 print(x);

```

12. Command-Line Interface

The CVM++ executable (`cvm.exe` on Windows) provides the following modes of operation:

Command	Description
<code>cvm compile src.cvm out.cvmb</code>	Compile source to a bytecode file
<code>cvm compile src.cvm out.cvmb --dis</code>	Compile and print a disassembly
<code>cvm run file.cvmb</code>	Load and execute a bytecode file
<code>cvm exec src.cvm</code>	Compile and run in a single step
<code>cvm exec src.cvm --debug</code>	Single-step with debug output
<code>cvm src.cvm</code>	Shorthand for <code>exec</code>
<code>cvm</code>	Launch the interactive REPL
<code>cvm help</code>	Display usage information

Table 3: CVM++ Command-Line Interface

12.1 Building the Executable

From the `src/` directory, run:

```

g++ -std=c++17 -Wall -o cvm lexer.cpp parser.cpp compiler.cpp
    vm.cpp bytecode_io.cpp main.cpp

```

No external libraries or build systems are required.

12.2 Interactive REPL

Running `cvm` with no arguments launches the Read-Eval-Print Loop, which allows code to be entered and executed interactively:

```
=====
CVM++ Interactive REPL
Type 'exit' to quit
=====
>> let x = 5;
>> print(x);
5
>> exit
```

13. Sample Program and Disassembly

13.1 Source Code

The following program prints the integers from 1 to 5:

```
1 let i = 1;
2 while (i < 6) {
3     print(i);
4     i = i + 1;
5 }
```

13.2 Compilation and Disassembly

Running `cvm compile program.cvm program.cvmb --dis` produces:

```
===== Disassembly =====
Variables (1):
  [0] i

Instructions (13):
  0  PUSH          1
  1  STORE          0 ; i
  2  LOAD           0 ; i
  3  PUSH          6
  4  LT
  5  JUMP_IF_FALSE  -> 13
  6  LOAD           0 ; i
  7  PRINT
  8  LOAD           0 ; i
  9  PUSH          1
 10  ADD
 11  STORE          0 ; i
 12  JUMP           -> 2
 13  HALT
=====
```

13.3 Execution Output

Running `cvm run program.cvmb` produces:

```
1
2
3
4
5
```

14. Challenges Encountered

- **Jump backpatching** — when compiling `if/else` and `while` constructs, the target address of a jump is not yet known at the time the jump instruction is emitted. The compiler must record the instruction's index and fill in the correct target once the block has been compiled.
- **Operator precedence** — the recursive descent parser handles precedence implicitly through the structure of its grammar functions. Ensuring correct precedence required careful ordering of parsing steps.
- **Stack discipline** — every instruction that consumes a value must be preceded by exactly the right number of pushes. Errors in stack depth manifest as runtime crashes rather than compile-time errors, making them harder to debug.
- **Binary file format design** — the `.cvmb` format must encode both the variable name table and the instruction stream in a way that allows exact reconstruction at load time, with no ambiguity in field boundaries.

15. Possible Future Improvements

- **Functions** — user-defined functions with parameters and return values, requiring a call stack.
- **Strings** — string literals and basic string operations.
- **Arrays** — fixed-size array support with index-based access.
- **Additional comparison operators** — `<=`, `>=`, `!=`.
- **Logical operators** — `and`, `or`, `not`.
- **Optimising compiler** — constant folding, dead code elimination.
- **Improved error reporting** — line and column numbers in error messages.
- **Debugger** — step-through execution and stack inspection.

16. Conclusion

CVM++ demonstrates the complete pipeline from source code to execution output in a single, self-contained C++ project. By building each component from scratch — the lexer, parser, compiler, bytecode serialiser, and virtual machine — the project makes concrete the processes that are ordinarily hidden inside production language runtimes.

The two-phase design, in which compilation and execution are separate operations, mirrors

the model used by languages such as Java (source to `.class` files, executed by the JVM) and Python (source to `.pyc` bytecode, executed by CPython). CVM++ demonstrates that this architecture, while seemingly complex, can be built from clearly separated, understandable components.